



UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – COMPUTAÇÃO GRÁFICA
PROF. MSc. RODRIGO LYRA

GUSTAVO HENRIQUE STAHL MÜLLER
PAULO HENRIQUE ROHLING

EXPLORANDO O PIPELINE DE RENDERIZAÇÃO DO OPENGL

ITAJAÍ
2022

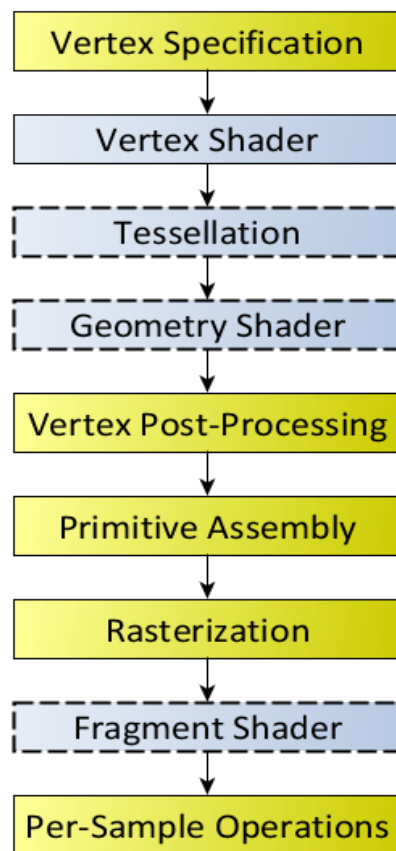
1. INTRODUÇÃO

O objetivo deste trabalho é explorar o pipeline de renderização do OpenGL, com o intuito de definir o que é o pipeline de renderização, quais são as etapas do pipeline, em que ordem cada etapa é processada, quais das etapas são utilizadas para a aplicação de *shaders* e qual é a diferença entre *Vertex Shader* e *Fragment Shader*.

2. DESENVOLVIMENTO

O OpenGL possui uma sequência de passos executados em ordem para renderizar objetos. Esta sequência é chamada de Pipeline de Renderização. O diagrama que ilustra as etapas do pipeline é mostrado na Figura 1. Cada etapa será explicada em sua ordem de execução nos tópicos a seguir.

Figura 1 – Diagrama do Pipeline de Renderização do OpenGL.



Fonte: OPENGGL, 2022

2.1. PIPELINE DE RENDERIZAÇÃO

2.1.1. Vertex Specification

Esta é a etapa onde é definida a forma como uma lista ordenada de vértices será carregada. Ela basicamente representa o vértice em *Vertex Array Objects* (que são os dados que o vértice contém) e *Vertex Buffer Objects* (que são os dados do próprio vértice).

Estes vértices, depois de especificados, serão renderizados na forma de uma *Primitive*, que é uma pequena forma básica definida pelo OpenGL, como triângulos, linhas e pontos. Este processo é chamado de *Vertex Rendering*.

2.1.2. Vertex Processing

Esta etapa é responsável por definir de que forma os vértices especificados na etapa anterior serão processados. Por este motivo ela se divide nas subetapas de *Vertex Shader*, *Tessellation* e *Geometry Shader*.

2.1.2.1. Vertex Shader

Esta subetapa aplica um processamento através de um shader definido em GLSL (programável pelo usuário) em cada vértice armazenado no VAO (*Vertex Array Object*) de maneira individual.

2.1.2.2. Tessellation

Esta é uma subetapa opcional responsável por dividir pequenas porções de vértices, intituladas de patch, em pequenas Primitives. Isto é feito em três estágios.

No primeiro estágio, chamado de *Tessellation Control Shader*, é feita uma proteção para que a quantidade de tesselação aplicada não quebre ou gere espaços entre os patches.

O segundo estágio é o *Tessellation Primitive Generator*, responsável por gerar novos patches com os novos valores do primeiro estágio.

O último estágio é o *Tessellation Evaluation Shader*, responsável por gerar novos vértices com os valores dos patches tesselados na etapa anterior.

2.1.2.3. Geometry Shader

Este é um processo opcional definido pelo usuário onde as *Primitives* de entrada são processadas, de forma que a saída pode ser zero ou mais *Primitives*. Logo, esta etapa pode tanto remover *Primitives* quanto tesselá-las, de forma que produza mais de uma *Primitive* seja produzida a partir de uma única entrada.

Além disso, existe a possibilidade de alterar a forma da *Primitive*. Por exemplo, seria possível converter um triângulo em linhas ou linhas em pontos.

2.1.3. Vertex Post-Processing

Esta etapa é responsável por definir alguns processamentos posteriores ao processamento baseado em shaders. Ela se divide nas subetapas *Transform Feedback*, *Primitive Assembly*, *Clipping* e *Face Culling*.

2.1.3.1. Transform Feedback

Esta subetapa representa as *Primitives* geradas pela etapa anterior na forma de *Buffer Objects*, que é o formato utilizado para fazer os processamentos das subetapas seguintes.

O *Buffer Object* basicamente é um array alocado para guardar dados diversos, tais como os dados dos vértices.

2.1.3.2. Primitive Assembly

Esta etapa divide o conjunto de *Primitives* gerado em uma sequência de *base primitives*. Este é um outro significado do termo *Primitive* utilizado pelo OpenGL, o que pode ser um pouco confuso. O termo *base primitive* é usado para se referir ao resultado da etapa anterior e só é utilizado nesta etapa.

2.1.3.3. Clipping

Nesta etapa, as primitivas que estão fora das bordas do volume de visualização são cortadas, de forma que todas as primitivas que serão passadas para as próximas etapas do pipeline estão dentro dos limites do volume de visualização.

2.1.3.4. Face Culling

Esta é uma etapa aplicada em primitivas triangulares para definir sua face aparente. Isso é feito utilizando a ordem dos três vértices que compõem o triângulo e permite que, com base na face aparente, outros triângulos possam ser descartados.

2.1.4. Rasterization

Detecta os pixels cobertos por uma primitiva, transformando ela em um ou mais fragmentos que serão usados pelas próximas etapas do pipeline.

2.1.5. Fragment Shader

Esta etapa processa cada um dos fragmentos gerados pela etapa de renderização, preenchendo cada um com as suas respectivas cores e interpolando entre cores caso os vértices de um mesmo fragmento possuam cores diferentes.

O *Fragment Shader* recebe como entrada um fragmento e produz como saída outro fragmento. Caso um *Fragment Shader* não seja definido, as cores finais dos fragmentos serão indefinidas.

2.1.6. Per-Sample Operations

Esta etapa passa por cada um dos fragmentos gerados pelo *Fragment Shader* por uma série de testes. Alguns desses testes são:

- a) ***Pixel Ownership Test***: Descarta fragmentos que usam pixels não possuídos pelo OpenGL, como pixels de janelas que estão sobrepondo a janela atual.
- b) ***Scissor Test***: O programador pode definir uma área do frame buffer como sendo a única área válida de renderização. Nesse caso, todos os fragmentos que possuem pixels fora dessa área são descartados.
- c) ***Stencil Test***: Permite descartar fragmentos com base na comparação entre o valor de *stencil* do fragmento e o valor *stencil* da posição do fragmento.

- d) **Depth Test:** Permite descartar fragmentos com base na comparação entre o valor de profundidade do fragmento e o valor de profundidade da posição do fragmento.

Por último, a mesclagem é feita, que mescla as propriedades dos pixels dos fragmentos com as propriedades dos pixels respectivos do *frame buffer* onde os fragmentos serão renderizados.

2.2. ETAPAS QUE ENVOLVEM SHADER

As três etapas que envolvem shaders são *Vertex Shader*, *Geometry Shader* e *Fragment Shader*, sendo que a etapa de *Geometry Shader* é opcional.

2.3. DIFERENÇA ENTRE VERTEX SHADER E FRAGMENT SHADER

A diferença entre *Vertex Shader* e *Fragment Shader* é que o *Vertex Shader* atua sobre um vértice, enquanto o *Fragment Shader* atua sobre um fragmento.

O papel do *Vertex Shader* é transformar a posição ou alguma outra propriedade de um vértice (como a normal), e o papel do *Fragment Shader* é transformar as propriedades dos pixels entre os vértices, e não os vértices em si (pois o *Vertex Shader* é responsável por isso e informações sobre vértices são perdidas depois da etapa de renderização).

REFERÊNCIAS

OPENGL. **Rendering Pipeline Overview**. Disponível em: <https://www.khronos.org/opengl/wiki/Rendering_Pipeline_Overview>. Acesso em: 16 mai. 2022.

SILVA, F. C. **Detalhamento de Superfícies Utilizando Tesselação em Hardware**. Capítulo 3. Disponível em: <https://web.tecgraf.puc-rio.br/~abraposo/pubs/alunos/dissertacao_Fabricio_Silva_ju12.pdf>. Acesso em: 16 mai. 2022.

WETZSTEIN, G. **The Graphics Pipeline and OpenGL**. Disponível em: <<https://stanford.edu/class/ee267/lectures/lecture2.pdf>>. Acesso em 16 mai. 2022.